

From Local LLM Inference to a Tool-Using Agent

A lossless textbook-style edition of the current chat thread

This edition preserves the complete visible user and assistant content from this thread, while adding section headings and layout only for readability. Code, terminal output, questions, corrections, and explanatory diagrams are retained.

Topics covered

Mutable `llama_context` state and return codes; how `llama_decode` updates the KV cache; empty context creation; the first agent/tool loop; dynamic tokenizer-buffer sizing; structured-output parsing; and the origin of Qwen's `<think>` blocks.

How to Read This Edition

The textbook is organized chronologically. Each exchange is retained as a source transcript. Formatting is additive: speaker labels and section titles are added, but the preserved transcript content itself is not summarized away.

Chapter 1 - Mutable Context and Native Memory

USER

and I am not using `decode_result`. then how is `ctx` stays updated in RAM ?

ASSISTANT

Exactly – `decode_result` does **not** contain the updated model state. It's only a **status/error code**.

Look at this:

```
```python id="wdsgvd"
decode_result = bridge.llama_bridge_decode_prompt(
 ctx,
 token_buffer,
 token_count,
)
...
```
```

There are two outputs happening here, but in different ways:

```
```text id="8iwox8"
 llama_decode(...)
 / | \
 v | v
 modifies ctx returns int
 IN PLACE
 |
 v
 KV cache etc.
 updated
...
 |
 v
 decode_result
 0 = success
...
```
```

So `decode_result` might simply be:

```
```python id="4af7bj"
0
...
```
```

which means:

```
> llama.cpp successfully processed the tokens.
```

That's why we only use it here:

```
```python id="owbz9x"
if decode_result != 0:
 raise RuntimeError(...)
...
```
```

The important object is:

```
```python id="asy4dz"
ctx
...
```
```

What is `ctx`?

Earlier we created it:

```

python id="azwugo"
ctx = bridge.llama_bridge_create_context(
    model,
    context_size,
)

```

Remember that we defined:

```

python id="khgirw"
bridge.llama_bridge_create_context.restype = ctypes.c_void_p

```

So Python's `ctx` is essentially a **pointer**.

Conceptually:

```

text id="54g0c7"
Python

```

```

ctx
|
| pointer
| 0x12345678
▼
_____ native memory _____

```

```

llama_context
|
| KV cache
| inference state
| model-related runtime state
| ...

```

Python isn't holding that entire structure.

It's holding something equivalent to:

```

text id="lh764t"
"the llama_context is over there → 0x12345678"

```

Then we pass that pointer into C++

Here:

```

python id="dkh6hd"
bridge.llama_bridge_decode_prompt(
    ctx,
    token_buffer,
    token_count,
)

```

C++ receives:

```

cpp id="gzl0hb"
llama_context * ctx

```

and eventually executes:

```

cpp id="o7t62b"
llama_decode(
    ctx,
    batch
);

```

`llama\_decode()` operates on that existing context.

So imagine before decoding:

```
```text id="2l9pqa"
ctx
|
▼
llama_context
KV cache: empty
...
```
```

We send:

```
```text id="64x46s"
The capital of France is
```
```

through `llama_decode()`.

Afterwards, the **same context object** might conceptually be:

```
```text id="4uqjfq"
ctx
|
▼
llama_context
KV cache:
The → K0, V0
capital → K1, V1
of → K2, V2
France → K3, V3
is → K4, V4
latest logits available
...
```
```

Your Python variable hasn't changed:

```
```python id="x0ni2t"
ctx
```
```

It still points to the **same native object**.

But the contents of the memory that it points to have changed.

This is classic pointer behavior

A tiny C analogy makes it clearer:

```
```c id="smostm"
void increment(int *x) {
 *x = *x + 1;
}
```
```

Suppose:

```
```c id="juks8"
int number = 5;

increment(&number);
```
```

The function doesn't need to return `6`.

It modified the memory directly:

```
```text id="021vme"
BEFORE
```



```
increment(&number)
```

```
AFTER
```



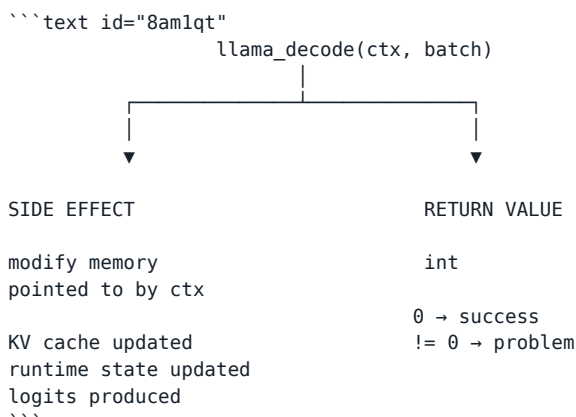
```
```
```

The return value could simply say whether the operation succeeded.

That's essentially what's happening with:

```
```cpp id="1vlg6g"
llama_decode(ctx, batch)
```
```

So there are two completely different channels



And this explains our generation loop.

First:

```
```python id="z5613z"
bridge.llama_bridge_decode_prompt(
 ctx,
 token_buffer,
 token_count,
)
```
```

modifies:

```
```text id="p26xuw"
ctx → [prompt state]
```
```

Then:

```

python id="82x68r"
next_token = bridge.llama_bridge_sample_greedy(ctx)

```

gets the logits associated with that context and chooses a token.

Then:

```

python id="wto46a"
bridge.llama_bridge_decode_one(
    ctx,
    next_token,
)

```

again modifies **the same context**:

```

text id="rjhkpw"
Before:

```

```

ctx
↓
[prompt]

```

```

decode_one(" Paris")

```

After:

```

ctx
↓
[prompt + Paris]

```

Then again:

```

text id="lv2fnf"
sample(ctx)
↓
"."

```

```

decode_one(ctx, ".")
↓

```

```

ctx
↓
[prompt + Paris + .]

```

That's the central idea:

```

text id="9raq6p"
                SAME ctx POINTER
                |
                ▼
Prompt —decode→ [context state]
                |
Paris —decode→ [context state + Paris]
                |
. —decode→ [context state + Paris + .]
                |
                ...

```

So you're right to notice that `decode_result` isn't being used for inference. **It isn't supposed to be.**

`ctx` is being mutated **in place in native memory**, while `decode_result` only tells Python whether that mutation/forward

USER

```


```

```
bridge.llama_bridge_decode_one(
    ctx,
    next_token,
)
```

How is next_token gets appened here, or batch gets appended in the other function ?
 ...

ASSISTANT

This is an important detail: `next_token` is not appended to a Python list, and `batch` itself is not appended to `ctx`.

The persistence happens `inside llama_decode()`, primarily through the KV cache.

Let's follow your exact code.

Your Python calls:

```
python id="zjl7wb"
bridge.llama_bridge_decode_one(
    ctx,
    next_token,
)
...
```

Suppose:

```
text id="gmpfis"
next_token = 12345 → " Paris"
...
```

Python passes two things to C++:

```
text id="v96cfd"
ctx → existing llama_context
next_token → 12345
...
```

Our C++ function is:

```
cpp id="a3z6j5"
int llama_bridge_decode_one(
    llama_context * ctx,
    int token
) {
    llama_token t = token;

    llama_batch batch =
        llama_batch_get_one(
            &t,
            1
        );

    return llama_decode(
        ctx,
        batch
    );
}
...
```

There are two separate stages here.

First: create a temporary batch

This:

```
cpp id="tgamvw"
llama_token t = token;
...
```

gives:

```
```text id="piwgk6"
t = 12345
```
```

Then:

```
```cpp id="3fwfrc"
llama_batch batch =
 llama_batch_get_one(&t, 1);
```
```

creates a batch describing:

```
```text id="is4vvh"
batch
```

token: 12345
count: 1

```
```
```

This batch is essentially **input for this forward pass**.

It is not the persistent history.

Then:

```
```cpp id="wwa9m6"
llama_decode(ctx, batch);
```
```

is where the important work happens.

What `llama_decode()` does

Suppose before this call we've already processed:

```
```text id="hv8dpg"
The capital of France is
```
```

The context has cached information for those positions:

```
```text id="rbsjn9"
ctx
```

KV cache

position 0	The	$K_0$ $V_0$
position 1	capital	$K_1$ $V_1$
position 2	of	$K_2$ $V_2$
position 3	France	$K_3$ $V_3$
position 4	is	$K_4$ $V_4$

Now we call:

```
```cpp id="9u7igv"
llama_decode(ctx, batch);
```
```

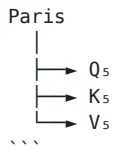
where:

```
```text id="lhtyyj"
batch = [Paris]
```
```

The model performs a forward pass for that new token.

At the attention layers it calculates the new token's key and value vectors:

```
```text id="zwwwbm"
```



The query `Q<sub>5</sub>` is used for the current attention calculation.

But importantly, `K<sub>5</sub>` and `V<sub>5</sub>` are stored in the context's KV cache.

So after `llama\_decode()`:

```
```text id="606tpj"
ctx
```

KV cache

position 0	The	K <sub>0</sub>	V <sub>0</sub>	
position 1	capital	K <sub>1</sub>	V <sub>1</sub>	
position 2	of	K <sub>2</sub>	V <sub>2</sub>	
position 3	France	K <sub>3</sub>	V <sub>3</sub>	
position 4	is	K <sub>4</sub>	V <sub>4</sub>	
position 5	Paris	K <sub>5</sub>	V <sub>5</sub>	← NEW

...

That's the "append" you're looking for.

But technically it's better not to think:

```
```text id="75guun"
ctx.text += "Paris"
```
```

because llama.cpp does not need to maintain your original text that way.

It maintains the **model state** needed to continue inference.

So when earlier I showed:

```
```text id="r92sdc"
ctx = [prompt + Paris]
```
```

that was conceptual shorthand.

More accurately:

```
```text id="txablk"
ctx
```

does NOT necessarily contain:

"The capital of France is Paris"

as a Python-style string/list.

Instead it contains runtime state such as:

```
KV cache
positions
sequence state
output/logit state
memory buffers
etc.
```
```

### And the same thing happened with the original prompt

Remember:

```
```python id="b2yigt"
```

```
bridge.llama_bridge_decode_prompt(
    ctx,
    token_buffer,
    token_count,
)
...

```

Our C++ code does:

```
```cpp id="fgm383"
llama_batch batch =
 llama_batch_get_one(
 tokens,
 token_count
);

return llama_decode(
 ctx,
 batch
);
```

```

Suppose tokenization gave:

```
```text id="ah3m88"
[The, capital, of, France, is]
```

```

Then the batch is approximately:

```
```text id="m12ps7"
batch

```

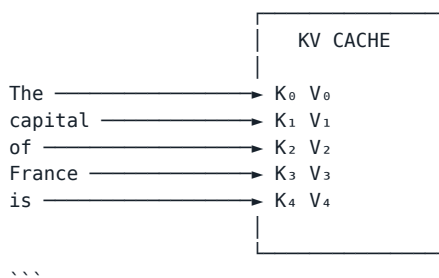
```
position 0 → The
position 1 → capital
position 2 → of
position 3 → France
position 4 → is
```

```

`llama\_decode()` processes them and populates the context:

```
```text id="ghdubn"
llama_context

```



Then we sample:

```
```text id="hln8xu"
logits
↓
Paris
```

```

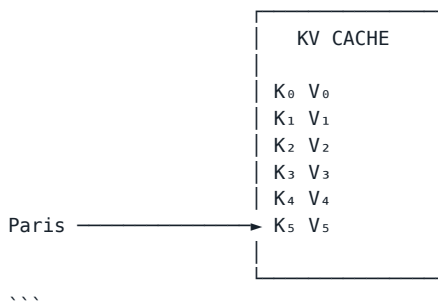
Then:

```
```python id="opjfx1"
decode_one(ctx, Paris)
```

```

causes:

```
```text id="8s2b4d"
```



Then sample again:

```
```text id="zby28m"
logits
 ↓
"."
```
```

Then:

```
```python id="06gjh4"
decode_one(ctx, ".")
```
```

results in another position being added:

```
```text id="5m87tl"
KV CACHE

position 0 The K0 V0
position 1 capital K1 V1
position 2 of K2 V2
position 3 France K3 V3
position 4 is K4 V4
position 5 Paris K5 V5
position 6 . K6 V6
```
```

So the key distinction is:

```
```text id="dwsul4"
batch
=
temporary input for THIS llama_decode() call
```

```
ctx / KV cache
=
persistent inference state across calls
```
```

And that's exactly why we don't need to repeatedly send:

```
```text id="vtbxbi"
The capital of France is Paris .
```
```

Instead:

```
```text id="5g563v"
First call:
```

```
batch = [The, capital, of, France, is]
```

↓  
KV cache  
 $K_0V_0 \dots K_4V_4$

Second call:

```
batch = [Paris]
```

↓

KV cache  
 $K_0V_0 \dots K_5V_5$

Third call:

```
batch = [.]
```

↓

KV cache  
 $K_0V_0 \dots K_6V_6$   
...

This **incremental decoding + KV caching** is one of the most important reasons autoregressive LLM inference can generate to

# Chapter 2 - Creating an Empty Inference Context

## USER

```
ctx = llama_bridge.llama_bridge_create_context(
 model,
 context_size,
)
```

does it create empty context ?

## ASSISTANT

Yes – **conceptually**, it creates an empty inference context

When you do:

```
```python id="9goo8p"  
ctx = llama_bridge.llama_bridge_create_context(  
    model,  
    context_size,  
)  
```
```

our C++ bridge does:

```
```cpp id="1mhd57"  
llama_context_params params =  
    llama_context_default_params();  
  
params.n_ctx = context_size;  
params.n_batch = context_size;  
  
return llama_init_from_model(  
    model,  
    params  
);  
```
```

At this point, think of the situation as:

```
```text id="l1748f"  
model  
|  
├─ Qwen weights  
├─ architecture  
├─ vocabulary  
└─ configuration  
  
ctx  
|  
├─ KV cache allocated      ← but no prompt tokens processed yet  
├─ inference buffers  
├─ context capacity  
└─ runtime state  
```
```

So if:

```
```python id="sjiafz"  
context_size = 512  
```
```

you can conceptualize it as creating space for a sequence:

```
```text id="kldxrc"
```

KV cache capacity

```
position
  0      empty
  1      empty
  2      empty
  3      empty
  ...
  511    empty
...
```

It does **not** yet contain:

```
```text id="592cto"
"The capital of France is"
```
```

because we haven't called ``llama_decode()``.

Then this happens:

```
```python id="f121q6"
bridge.llama_bridge_decode_prompt(
 ctx,
 token_buffer,
 token_count,
)
```
```

Suppose the prompt has 5 tokens.

Before:

```
```text id="hvjrct"
ctx

position 0 empty
position 1 empty
position 2 empty
position 3 empty
position 4 empty
...
```
```

After decoding:

```
```text id="velvbb"
ctx

position 0 K0 V0 ← "The"
position 1 K1 V1 ← "capital"
position 2 K2 V2 ← "of"
position 3 K3 V3 ← "France"
position 4 K4 V4 ← "is"

position 5 empty
position 6 empty
...
```
```

Then suppose the model predicts ``" Paris"``.

We do:

```
```python id="vl9j5g"
bridge.llama_bridge_decode_one(
 ctx,
 next_token,
)
```
```

and now:

```

```text id="f6h3oe"
position 0 K0 V0
position 1 K1 V1
position 2 K2 V2
position 3 K3 V3
position 4 K4 V4
position 5 K5 V5 ← " Paris"

position 6 empty
...

```

One technical correction to the mental model, though: `context\_size = 512` doesn't necessarily mean llama.cpp literally allo

But for understanding the agent we're building, this is a good mental model:

```

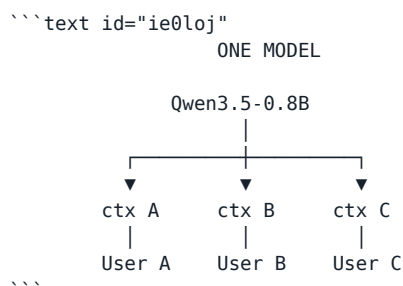
```text id="bauimq"
llama_model
=
long-lived, read-only-ish model weights
"what Qwen knows"

llama_context
=
one inference/conversation's working state
"what Qwen has processed so far"
...

```

And this distinction will become **very important for agents**.

Eventually you could have:



You don't need three copies of the model weights just because you have three independent inference sequences. Each sequence

So yes: **llama_bridge_create_context()** creates the inference context before any prompt has been decoded into it.

Chapter 3 - Turning the Local LLM into an Agent

USER

ok, now lets do the next step to make the llm an agent

ASSISTANT

Now we'll add the **agent loop**, not more inference machinery. The LLM will choose between "answer normally" and "call `sim`"

ASSISTANT

Yes. We now have enough infrastructure to build the first real agent loop.

For the **first agent version**, I recommend we do not depend on llama.cpp's built-in tool-calling format yet. Instead, we d

```
```text id="ae0pjt"
```

If you want to call a tool, output:

```
{"action":"tool","name":"simple_test","arguments":{}}
```

If you want to answer the user, output:

```
{"action":"answer","content":"..."}
```
```

That keeps the agent logic completely ours. Later we can switch to the model's native chat/tool template without changing th

Your flow becomes:

```
```text id="lglare"
```

User

|

▼  
Agent

| gives model:  
| - user request  
| - available tools

▼

Qwen

| answer normally

| request tool

|

▼  
simple\_test()

|

▼  
tool result

|

▼  
Qwen again

|

▼  
final answer

...

## 1. Create `skills.py`

Put this in `skills.py`:

```
```python id="flxges"
def simple_test() -> str:
    return "Simple test"
```
```

Nothing AI-related here.

It's just ordinary Python.

## 2. Create a tool registry

Now create `tools.py`:

```
```python id="ta2aep"
from skills import simple_test
```

```
TOOLS = {
    "simple_test": simple_test,
}
```
```

This gives us:

```
```text id="iniln6"
tool name
  ↓
"simple_test"
  ↓
Python function
  ↓
simple_test
```
```

Later this can grow:

```
```python id="ifqq7k"
TOOLS = {
    "simple_test": simple_test,
    "read_file": read_file,
    "run_sql": run_sql,
    "deploy_bigquery": deploy_bigquery,
}
```
```

## 3. Turn your inference code into `generate()`

Until now `inference.py` runs immediately.

We want the agent to be able to do:

```
```python id="5oeiy9"
response = model.generate(prompt)
```
```

So conceptually restructure it into:

```
```python id="psdvq0"
class LocalLLM:

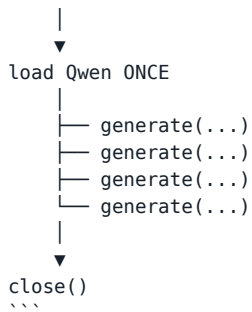
    def __init__(self):
        # load llama.cpp
        # load model

    def generate(self, prompt: str) -> str:
        # create context
        # tokenize
        # decode prompt
        # generate tokens
        # return text

    def close(self):
        # free model
        # shutdown llama.cpp
```
```

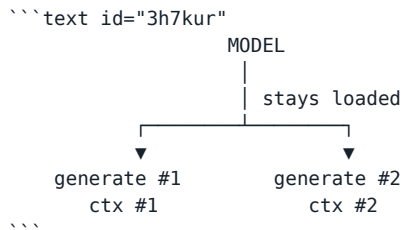
The important lifecycle becomes:

```
```text id="obhvjb"
LocalLLM()
```



Notice that I'm suggesting a ****new context for each complete `generate()` call****, while keeping the model loaded.

For now that's much easier to reason about:



We'll optimize context reuse later.

4. The actual agent

Now create:

```

```text id="3h7kur"
agent.py
```

```

Start with:

```

```python id="64p15n"
import json

from inference import LocalLLM
from tools import T00LS
```

```

Then define the system instructions:

```

```python id="sbwe7p"
SYSTEM_PROMPT = """
You are an agent.

```

You have access to these tools:

1. `simple_test`  
Description: Runs a simple test.  
Arguments: none.

You must respond using valid JSON only.

If you need to call a tool:

```

{
 "action": "tool",
 "name": "simple_test",
 "arguments": {}
}

```

If you want to answer the user:

```

{
 "action": "answer",
 "content": "your answer here"
}

```

```
}
```

```
Do not output anything outside the JSON.
"""
```
```

Then the core agent loop:

```
```python id="yco19i"
def run_agent(
 llm: LocalLLM,
 user_message: str,
) -> str:

 conversation = f"""
{SYSTEM_PROMPT}

USER:
{user_message}

ASSISTANT:
"""

 while True:

 # -----
 # Ask the LLM what to do
 # -----

 response = llm.generate(conversation)

 print("LLM OUTPUT:")
 print(response)

 decision = json.loads(response)

 # -----
 # Normal answer
 # -----

 if decision["action"] == "answer":

 return decision["content"]

 # -----
 # Tool call
 # -----

 if decision["action"] == "tool":

 tool_name = decision["name"]

 arguments = decision.get(
 "arguments",
 {},
)

 # Find Python function
 tool = TOOLS.get(tool_name)

 if tool is None:
 raise RuntimeError(
 f"Unknown tool: {tool_name}"
)

 # Execute Python function
 tool_result = tool(
 **arguments
)
```

```

 print(
 f"TOOL RESULT: {tool_result}"
)

 # -----
 # Give result back to LLM
 # -----

 conversation += f"""
{response}

TOOL RESULT:
{tool_result}

Now decide what to do next.

ASSISTANT:
"""

 continue

 raise RuntimeError(
 f"Unknown action: {decision}"
)
 ...

Finally:

```python id="ltawqc"
def main():

    llm = LocalLLM()

    try:

        result = run_agent(
            llm,
            "Please run the simple test.",
        )

        print("\nFINAL ANSWER:")
        print(result)

    finally:

        llm.close()

if __name__ == "__main__":
    main()
...

## What should happen?

User says:

```text id="u77c6i"
Please run the simple test.
```

First LLM call:

```text id="3msx03"
User request
|
▼
Qwen
```

The model should produce:

```

```

```json id="4ihyij"
{
 "action": "tool",
 "name": "simple_test",
 "arguments": {}
}
```

```

Python parses:

```

```python id="fgwwdc"
decision = json.loads(response)
```

```

Then:

```

```python id="khmbif"
tool_name = decision["name"]
```

```

gives:

```

```text id="fwjnwz"
simple_test
```

```

Then:

```

```python id="336hd4"
tool = T00LS[tool_name]
```

```

gives the actual Python function:

```

```python id="bf40sy"
simple_test
```

```

Then:

```

```python id="auyi5g"
tool_result = tool()
```

```

actually runs:

```

```python id="nm8xci"
def simple_test() -> str:
 return "Simple test"
```

```

Result:

```

```text id="qixsbo"
Simple test
```

```

Then we send that result back to Qwen:

```

```text id="srmnl8"
User:
Please run the simple test.

```

```

Assistant:
{"action": "tool", "name": "simple_test", "arguments": {}}

```

```

Tool:
Simple test

```

```

Assistant:
```

```

The second LLM call should produce something like:

```

```json id="tcbg49"
{
 "action": "answer",
 "content": "The simple test completed successfully."
}
```

```

Python sees:

```

```python id="vuire7"
if decision["action"] == "answer":
 return decision["content"]
```

```

and the agent finishes.

So this loop:

```

```python id="2o8xv7"
while True:
```

```

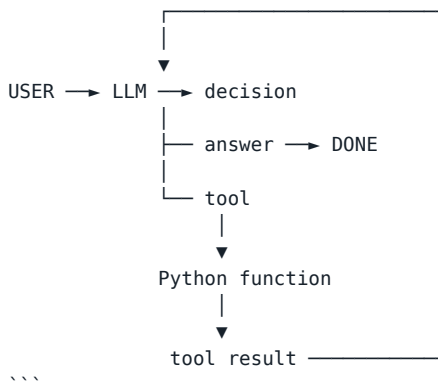
is actually the heart of your agent.

It is essentially:

```

```text id="2yfnx3"

```



That is the key transition from:

```

```text id="fu0evx"
LLM
=
text generator
```

```

to:

```

```text id="8t7lk1"
Agent
=
LLM
+
tools
+
execution loop
+
state
```

```

One important improvement should come immediately after we get this working: instead of manually writing pseudo-chat text li

But I would first get this minimal agent loop working. Then the next step should be **proper chat templating**, because that

# Chapter 4 - Debugging Tokenization Capacity

## USER

```
Model loaded successfully.
Model size: 526.50 MiB
Parameters: 752,393,024
```

```
Model unloaded.
```

```
Traceback (most recent call last):
```

```
File "/Users/farzananitol/football-fan-ex/llm_server/src/ai/agents/simple_agent.py", line 103, in <module>
 main()
```

```
File "/Users/farzananitol/football-fan-ex/llm_server/src/ai/agents/simple_agent.py", line 92, in main
 result = run_agent(
 ^^^^^^^^^
```

```
File "/Users/farzananitol/football-fan-ex/llm_server/src/ai/agents/simple_agent.py", line 34, in run_agent
 response = llm.generate(conversation)
 ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/Users/farzananitol/football-fan-ex/llm_server/src/ai/engine/llama/inference.py", line 75, in generate
 raise RuntimeError("Failed to tokenize prompt")
```

```
RuntimeError: Failed to tokenize prompt
```

```
import ctypes
```

```
import sys
```

```
from pathlib *import* Path
```

```
ai/engine/llama/inference.py -> parents[2] == ai/*
```

```
_AI_DIR = Path(__file__).resolve().parents[2]
```

```
if str(_AI_DIR) not in sys.path:
```

```
 sys.path.insert(0, str(_AI_DIR))
```

```
from config.config *import* LLAMA_LIBRARY_PATH, LLAMA_BRIDGE_PATH, MODEL_PATH
```

```
from config.model_config.llama *import* MAX_TOKENS, CONTEXT_SIZE, MAX_NEW_TOKENS
```

```
from utils.llama_utils *import* define_llama_bridge_signatures, init_llama
```

```
class localLLM:
```

```
 def __init__(*self*):
```

```
 print(f"llama.cpp library: {LLAMA_LIBRARY_PATH}")
```

```
 self.llama_lib = ctypes.CDLL(LLAMA_LIBRARY_PATH)
```

```
 print("Successfully loaded llama.cpp into Python.")
```

```
 self.llama_bridge = ctypes.CDLL(LLAMA_BRIDGE_PATH)
```

```
 print("Successfully loaded llama.cpp bridge into Python.")
```

```
-----*
```

```
Define C function signatures*
```

```
-----*
```

```
 self.llama_bridge = define_llama_bridge_signatures(self.llama_bridge)
```

```
-----*
```

```
Initialize llama.cpp*
```

```
-----*
```

```

 self.llama\bridge = init_llama(self.llama\bridge)

 ## -----*

 ## Load model*

 ## -----*

 print("Loading model:")

 print(MODEL_PATH)

 self.model = self.llama\bridge.llama\bridge_model_load(

 str(MODEL_PATH).encode("utf-8")

)

 if* not self.model:

 raise* RuntimeError("Failed to load model")

 print("\nModel loaded successfully.")

 ## -----*

 ## Inspect model*

 ## -----*

 size_bytes = self.llama\bridge.llama\bridge_model_size(self.model)

 parameter_count = self.llama\bridge.llama\bridge_model_n_params(self.model)

 print(f"Model size: {size_bytes / 1024**2:.2f} MiB")

 print(f"Parameters: {parameter_count:,}")

 def generate(*self*, *prompt*: str) -> str:

 ## -----*

 ## Tokenize prompt*

 ## -----*

 token_buffer = (ctypes.c_int * MAX_TOKENS)()

 token_count = self.llama\bridge.llama\bridge_tokenize(

 self.model,

 prompt.encode("utf-8"),

 token_buffer,

 MAX_TOKENS

)

 if* token_count < 0:

 raise* RuntimeError("Failed to tokenize prompt")

 tokens = [token_buffer[i] *for* i *in* range(token_count)]

 print(f"\nText: {prompt}")

 print(f"Token count: {token_count}")

 print(f"Token IDs: {tokens}")

 ## -----*

```

```

Create inference context

-----

 context_size = CONTEXT_SIZE

 ctx = self.llama_bridge.llama_bridge_create_context(

 self.model,

 context_size,

)

if not ctx:

raise RuntimeError("Failed to create llama context")

 print("\nContext created.")

-----

Run the transformer

-----

 decode_result = self.llama_bridge.llama_bridge_decode_prompt(

 ctx,

 token_buffer,

 token_count,

)

if decode_result != 0:

raise RuntimeError(

 f"llama_decode failed: {decode_result}"

)

 print("Prompt decoded by neural network.")

-----

Generate multiple tokens

-----

 generated_text = ""

for _ *in* range(MAX_NEW_TOKENS):

1. Sample from the logits produced

by the most recent llama_decode()

 next_token = self.llama_bridge.llama_bridge_sample_greedy(

 ctx

)

2. Stop if model generated an

end-of-generation token

if self.llama_bridge.llama_bridge_is_eog(

 self.model,

```

```

 next_token,
):
break

3. Convert token ID → text*

 piece_buffer = ctypes.create_string_buffer(256)

 piece_length = self.llama_bridge.llama_bridge_token_to_piece(
 self.model,
 next_token,
 piece_buffer,
 len(piece_buffer),
)

 if piece_length < 0:
raise RuntimeError(
 f"Piece buffer too small: {-piece_length}"
)

 piece = piece_buffer.raw[:piece_length].decode(
 "utf-8",
errors="replace",
)

 generated_text += piece

 print(
 piece,
end="",
flush=True,
)

4. Feed the generated token back*
through the model*

 decode_result = self.llama_bridge.llama_bridge_decode_one(
 ctx,
 next_token,
)

 if decode_result != 0:
raise RuntimeError(
 f"llama_decode failed: {decode_result}"
)

 print("\n\nGeneration complete.")

 print(f"Generated text: {generated_text!r}")

```

```

 self.llama_bridge.llama_bridge_free_context(ctx)

return generated_text

 def close(*self*):

-----

Cleanup

-----

 self.llama_bridge.llama_bridge_model_free(self.model)

 self.llama_bridge.llama_bridge_shutdown()

 print("\nModel unloaded.")

```

## ASSISTANT

The failure is happening exactly here:

```

```python id="9c10zi"
token_count = self.llama_bridge.llama_bridge_tokenize(
    self.model,
    prompt.encode("utf-8"),
    token_buffer,
    MAX_TOKENS
)
```

```

and then:

```

```python id="8xnjcz"
if token_count < 0:
    raise RuntimeError("Failed to tokenize prompt")
```

```

The key point is: **\*\*a negative return from `llama\_tokenize()` usually does not mean tokenization failed. It means your output**

Previously your prompt was tiny:

```

```text id="03q79i"
The capital of France is
```

```

Now your agent prompt contains all of this:

```

```text id="8ucw50"
system instructions
tool descriptions
JSON examples
user message
ASSISTANT:
```

```

So `MAX\_TOKENS` is no longer large enough.

For example, if llama.cpp returns:

```

```text id="zgqffp"
-173
```

```

it means:

> I need space for 173 tokens, but the supplied buffer isn't large enough.

So first, change your error to see the real number:

```

```python id="7kjuom"
if token_count < 0:

```

```

        raise RuntimeError(
            f"Token buffer too small. Required {-token_count} tokens, "
            f"but MAX_TOKENS={MAX_TOKENS}"
        )
    ...

```

But I would not solve this simply by increasing `MAX\_TOKENS`.

Instead, remove the fixed tokenization buffer entirely and use llama.cpp's intended two-pass approach.

Fix `generate()`

Replace:

```

```python id="mk19md"
token_buffer = (ctypes.c_int * MAX_TOKENS)()

token_count = self.llama_bridge.llama_bridge_tokenize(
 self.model,
 prompt.encode("utf-8"),
 token_buffer,
 MAX_TOKENS
)

```

```

if token_count < 0:
 raise RuntimeError("Failed to tokenize prompt")
...

```

with:

```

```python id="25yqvt"
# First attempt with no output buffer.
#
# llama_tokenize() will return:
#
#   -N
#
# where N is the number of tokens required.

required = self.llama_bridge.llama_bridge_tokenize(
    self.model,
    prompt.encode("utf-8"),
    None,
    0,
)

if required >= 0:
    raise RuntimeError(
        f"Unexpected tokenization result: {required}"
    )

required = -required

print(f"Required token capacity: {required}")

# Now allocate exactly enough memory.

token_buffer = (ctypes.c_int * required)()

token_count = self.llama_bridge.llama_bridge_tokenize(
    self.model,
    prompt.encode("utf-8"),
    token_buffer,
    required,
)

if token_count < 0:
    raise RuntimeError(
        f"Tokenization failed. Required {-token_count} tokens."
    )
...

```

Your existing C++ function should work with this pattern:

```
```cpp id="e14hz0"
int llama_bridge_tokenize(
 llama_model * model,
 const char * text,
 int * tokens,
 int max_tokens
) {
 const llama_vocab * vocab =
 llama_model_get_vocab(model);

 int text_length =
 static_cast<int>(strlen(text));

 return llama_tokenize(
 vocab,
 text,
 text_length,
 tokens,
 max_tokens,
 true,
 true
);
}
```
```

When Python passes:

```
```python id="z6cymq"
None
```
```

for:

```
```python id="k0ao19"
ctypes.POINTER(ctypes.c_int)
```
```

ctypes sends a C null pointer:

```
```text id="ciaots"
nullptr
```
```

So the first call is effectively asking:

```
```text id="9m2sli"
How many tokens would you need?
```
```

Then the second call says:

```
```text id="tn9b70"
Okay, here's exactly that much memory.
Now tokenize it.
```
```

The flow becomes:

```
```text id="mcazka"
Agent prompt
|
▼
llama_tokenize(
 tokens=null,
 capacity=0
)
|
▼
returns -173
|
▼
```

Python allocates  
173 integers

```
|
▼
llama_tokenize(
 buffer,
 capacity=173
)
|
▼
returns 173
...
```

There's also a **\*\*second issue you should fix now\*\***.

You currently have:

```
```python id="7bh5yf"  
context_size = CONTEXT_SIZE  
```
```

Suppose:

```
```text id="kwmwex"  
prompt = 400 tokens  
MAX_NEW_TOKENS = 128  
CONTEXT_SIZE = 512  
```
```

Then your model needs roughly:

```
```text id="gcalpt"  
400 prompt tokens  
+  
128 generated tokens  
=  
528  
```
```

but the context only has room for:

```
```text id="bndudr"  
512  
```
```

So before creating the context, add:

```
```python id="us8b5v"  
required_context_size = (  
    token_count  
    + MAX_NEW_TOKENS  
)  
  
if required_context_size > CONTEXT_SIZE:  
    raise RuntimeError(  
        f"Context too small. "  
        f"Prompt={token_count}, "  
        f"generation={MAX_NEW_TOKENS}, "  
        f"required={required_context_size}, "  
        f"configured={CONTEXT_SIZE}"  
    )  
```
```

Or for this small experiment, dynamically choose:

```
```python id="apjxr9"  
context_size = max(  
    CONTEXT_SIZE,  
    token_count + MAX_NEW_TOKENS,  
)  
```
```

So your beginning of ``generate()`` becomes:

```

```python id="5op43u"
def generate(self, prompt: str) -> str:

    # -----
    # Determine required token capacity
    # -----

    required = self.llama_bridge.llama_bridge_tokenize(
        self.model,
        prompt.encode("utf-8"),
        None,
        0,
    )

    if required >= 0:
        raise RuntimeError(
            f"Unexpected tokenizer result: {required}"
        )

    required = -required

    # -----
    # Allocate exact token buffer
    # -----

    token_buffer = (
        ctypes.c_int * required
    )()

    token_count = self.llama_bridge.llama_bridge_tokenize(
        self.model,
        prompt.encode("utf-8"),
        token_buffer,
        required,
    )

    if token_count < 0:
        raise RuntimeError(
            f"Failed to tokenize. "
            f"Required {-token_count} tokens."
        )

    print(f"Token count: {token_count}")

    # -----
    # Create sufficiently large context
    # -----

    context_size = max(
        CONTEXT_SIZE,
        token_count + MAX_NEW_TOKENS,
    )

    print(
        f"Context size: {context_size}"
    )

    ctx = self.llama_bridge.llama_bridge_create_context(
        self.model,
        context_size,
    )
...

```

And the reason you saw:

```

```text id="r8rylt"
Model unloaded.
Traceback...
```

```

is almost certainly because your `simple_agent.py` has something like:

```
```python id="m1wn1l"
try:
 ...
finally:
 llm.close()
```
```

The exception occurs during `generate()`, then Python executes `finally`, so:

```
```python id="lher6j"
llm.close()
```
```

unloads the model, and only afterward prints the traceback.

That part is behaving correctly.

The immediate problem is simply that your agent prompt is ****much** larger than the little test prompts we used while building

Chapter 5 - Parsing Model Output Safely

USER

LLM OUTPUT:

<think>

</think>

```
{
  "action": "tool",
  "name": "simple_test",
  "arguments": {}
}
```

Model unloaded.

Traceback (most recent call last):

```
File "/Users/farzananitol/football-fan-ex/llm_server/src/ai/agents/simple_agent.py", line 103, in <module>
    main()
File "/Users/farzananitol/football-fan-ex/llm_server/src/ai/agents/simple_agent.py", line 92, in main
    result = run_agent(
             ^^^^^^^^^
File "/Users/farzananitol/football-fan-ex/llm_server/src/ai/agents/simple_agent.py", line 39, in run_agent
    decision = json.loads(response)
               ^^^^^^^^^^^^^^^^^^^
File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/json/__init__.py", line 346, in loads
    return _default_decoder.decode(s)
           ^^^^^^^^^^^^^^^^^^^^^^^^
File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/json/decoder.py", line 337, in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/json/decoder.py", line 355, in raw_decode
    raise JSONDecodeError("Expecting value", s, err.value) from None
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

ASSISTANT

Your agent logic is basically working. The model correctly chose the tool.

The failure is only because the model returned extra text before the JSON:

```
```text id="pizzcn"
<think>

</think>

{
 "action": "tool",
 "name": "simple_test",
 "arguments": {}
}
```
```

But this line expects the **entire string** to be valid JSON:

```
```python id="5uc9w6"
decision = json.loads(response)
```
```

`json.loads()` sees the first character:

```
```text id="ykb7z3"
<
```
```

instead of:

```
```text id="7ml34y"
{
```
```

so it fails immediately.

For now, parse only the JSON object from the response:

```
```python id="wsp4ju"
def extract_json(response: str) -> dict:
 start = response.find("{")
 end = response.rfind("}")

 if start == -1 or end == -1:
 raise RuntimeError(
 f"No JSON object found in model response:\n{response}"
)

 json_text = response[start:end + 1]

 return json.loads(json_text)
```
```

Then change:

```
```python id="s32y1k"
decision = json.loads(response)
```

to:

```python id="mt3mzo"
decision = extract_json(response)
```
```

So your `simple\_agent.py` becomes roughly:

```
```python id="v18r36"
import json

def extract_json(response: str) -> dict:
 start = response.find("{")
 end = response.rfind("}")

 if start == -1 or end == -1:
 raise RuntimeError(
 f"No JSON object found in model response:\n{response}"
)

 json_text = response[start:end + 1]

 return json.loads(json_text)
```
```

and inside the loop:

```
```python id="z6ojpc"
response = llm.generate(conversation)

print("LLM OUTPUT:")
print(response)

decision = extract_json(response)
```
```

Then your current output:

```
```text id="dtzclh"
<think>

</think>

{
 "action": "tool",
 "name": "simple_test",
 "arguments": {}
}
```

```
}
...
```

will become:

```
```python id="f52vnc"  
decision = {  
    "action": "tool",  
    "name": "simple_test",  
    "arguments": {},  
}  
...
```

and then this part can execute:

```
```python id="cc7m85"  
tool_name = decision["name"]

tool = T00LS.get(tool_name)

tool_result = tool()
...
```

which should give:

```
```text id="2znhbm"  
Simple test  
...
```

There is an important architectural point here, though. We have just discovered our **first real agent-runtime responsibility**

```
```text id="izpptk"  
LLM output is probabilistic text
|
▼
Agent runtime must interpret it safely
|
▼
structured action
|
▼
execute tool
...
```

The LLM said:

```
```text id="0eiqg5"  
"Use valid JSON only"  
...
```

but still produced:

```
```text id="b4maz0"  
<think>...</think>
...
```

A small model may not perfectly follow formatting instructions every time. So production agent code should never blindly assume

```
```python id="ti2jox"  
json.loads(llm_output)  
...
```

will always work.

For our learning version, `extract_json()` is a good next step.

One caution: `find("{") + rfind("}")` is deliberately simple. Later, if the model writes multiple JSON objects or braces in

But right now your first tool-selection step **already succeeded**:

```
```text id="0ky7ji"  
USER
|
```

```
▼
Qwen
|
▼
"call simple_test"
|
▼
Python parser ← this is where we currently failed
|
▼
TOOLS["simple_test"]
|
▼
simple_test()
```
```

So you're only one small parsing fix away from seeing the tool actually execute.

Chapter 6 - Understanding the <think> Block

USER

Ok, nice. But where do we get this <think> block suddenly ? It was not there when llm was not an agent

ASSISTANT

Exactly – and importantly, **the <think> block is not appearing because the LLM has somehow become an agent**.

It appeared because we changed the **prompt** we send to Qwen.

Previously you were doing something simple like:

```
```text id="j5kl73"
The capital of France is
```
```

Now we're sending something much more instruction-like:

```
```text id="tzklas"
You are an agent.
```

```
You have access to these tools:
...
```

You must respond using valid JSON only.

```
If you need to call a tool:
{
 "action": "tool",
 ...
}
```

```
USER:
Please run the simple test.
```

```
ASSISTANT:
```
```

That changes how Qwen3.5 behaves.

Where <think> actually comes from

Your Qwen model was trained to support a reasoning/thinking mode. For some prompts, it may generate special-looking text such as

```
```text id="urt3oo"
<think>
...
</think>
```
```

followed by the answer:

```
```json id="ulrtf5"
{
 "action": "tool",
 "name": "simple_test",
 "arguments": {}
}
```
```

These are still just **tokens generated by the model**.

Nothing in our Python agent code added:

```
```text id="zmmtn5"
<think>
```
```

And nothing in `llama.cpp` suddenly created an agent reasoning system.

The generation is still exactly what we built:

```
```text id="zjlxk1"
prompt
↓
llama_decode()
↓
logits
↓
greedy sampler
↓
token
↓
token
↓
token
↓

"<think>"
"\n"
...
"</think>"
"\n"
"{
...
```
```

Why didn't we see it before?

Because before, our prompts were more like raw completion:

```
```text id="uovs5p"
The capital of France is
```
```

The likely continuation is simply:

```
```text id="kgcnyl"
Paris
```
```

There isn't much reason for the model to enter its instruction/reasoning behavior.

Now we gave it a task requiring a decision:

```
```text id="7eelwi"
User wants something.
```

```
Available tools:
 simple_test
```

```
Should I:
 answer directly?
 or call a tool?
```

```
Output the appropriate action.
```
```

That is much closer to the kind of instruction/reasoning data Qwen was trained on.

So the model produced its thinking markers before its final structured response.

But there's another important issue in our implementation

We're currently manually constructing:

```
```python id="5f827d"
conversation = f"""
{SYSTEM_PROMPT}
```

```
USER:
```

```
{user_message}
```

```
ASSISTANT:
""
``
```

This is **not** yet the proper Qwen chat format.

Qwen's tokenizer/model has a specific chat template that determines how things such as:

```
``text id="1ybjl4"
system message
user message
assistant message
tool-related instructions
thinking behavior
``
```

should be represented as tokens.

Right now we're bypassing that and essentially saying:

> Here's a big piece of plain text containing the words USER and ASSISTANT. Figure it out.

And surprisingly, your 0.8B model did figure it out:

```
``json id="o2ra9m"
{
 "action": "tool",
 "name": "simple_test",
 "arguments": {}
}
``
```

That's a good sign, but it isn't the final architecture I'd want us to keep.

### This is actually the perfect time for our next step

We should **not** just hack away `<think>` forever with:

```
``python id="5yywow"
response.find("{")
``
```

That was useful to prove the loop works.

Now that the proof works, we should implement the model's proper chat template.

Our stack should evolve from:

```
``text id="kh8rjs"
Python strings
```

```
SYSTEM:
...
USER:
...
ASSISTANT:
```

↓

tokenize

↓

```
Qwen
``
```

to:

```
``text id="z26545"
Python messages
```

```
[
 {"role": "system", ...},
 {"role": "user", ...}
]
```

↓

Qwen chat template

↓

correct model-specific prompt

↓

tokenize

↓

Qwen

```

Then we'll deliberately decide what to do about **thinking mode** rather than accidentally encountering it.

One more distinction worth keeping very clear:

```text id="5evrlm"

LLM reasoning capability      ← belongs to Qwen

Tool definitions               ← supplied by us

Tool-selection decision       ← made by Qwen

Tool execution               ← performed by Python

Agent loop                   ← written by us

```

So `<think>` belongs to the **model-output side**, not to the agent machinery itself.

And yes—**proper chat templating is the next thing I would implement before adding more tools**. It will give us a much clearer

Appendix - Request for This Lossless Textbook

USER

Now without ommiting a single character in this chat thread, give me a nice pdf containing nicely formatted textbook prepare